App Dev Project Report

1. Student Details

Name: Arpit Benjamin
Roll Number: 25F1002206
Email: 25f1002206@ds.study.iitm.ac.in

About Me: I'm a student at IITM BS degree, I'm also pursuing BTECH from Birla Institute of Technology, Mesra.

2. Project Details

Project Title: Hospital Management System (HMS)

Problem Statement

Hospitals often rely on manual labor or outdated software to manage patients, doctors, appointments, and treatment history. This leads to conflicts in timings, difficulty in tracking records and appointments, poor transparency and management, poor healthcare services and time-consuming administrative work.

The goal of this project is to build a web-based Hospital Management System that enables Admins, Doctors, and Patients to interact with the system based on their roles, ensuring efficient and elegant appointment handling, patient history management, and secure access to hospital data.

Approach

The HMS application was built using the **Flask** framework with a clean modular architecture:

- The database and all tables (Admin, Doctor, Patient, Appointment, DoctorAvailability, Treatment, Department, etc.) are created **programmatically** using SQLAlchemy models. User roles (Admin, Doctor, Patient) have separate dashboards and permissions.
- Both backend and frontend validations were implemented.
- Dynamic routing, searching, appointment conflict checks, availability scheduling, treatment history, and status updates (Booked/Completed/Cancelled) were implemented.
- Styling and UI are done using **HTML, CSS, Jinja2 templates** and custom JS.

3. AI/LLM Declaration

I used ChatGPT (GPT-5) to assist with

- Writing SQLAlchemy model structure and generation of sample data for testing.
- Designing basic API endpoints for quick start.
- Improving naming and code organization to maintain consistency.
- Small UI/UX improvements and changes for Jinja2 templates.

The extent of AI/LLM usage is around **15%**, limited to code suggestions and documentation structure. **All final logic, debugging, database design, implementation, and testing were done manually.**

4. Technologies and Frameworks Used

- **Flask** – Core backend framework used to handle routing, requests, and application logic.
- **Jinja2** – Template engine for rendering dynamic HTML pages.
  SQLite – Lightweight local relational database used for storing all application data.
- **SQLAlchemy** – ORM used to define database models and generate tables programmatically.
- **HTML, CSS, JavaScript** – Frontend technologies for page structure, styling, and interactive client-side behavior.

5. Database Schema / ER Diagram

Table	Purpose	Fields
Admin	Stores admin user details	id, name, email, password, dob, contact, is_blacklisted
Department	Hospital departments	id, name, description
Doctor	Doctor details + linked department	id, name, email, password, dob, contact, is_blacklisted, specialization, department_id
Patient	Patient profiles	id, name, email, password, dob, contact, is_blacklisted
Appointment	Records patient–doctor appointments	id, patient_id, doctor_id, date, time, status, visit_type, test_done, original_date, original_time, reschedule_count
Treatment	One treatment linked to one appointment	id, appointment_id, diagnosis, prescription, notes, created_at, updated_at

Medicine	Medicines prescribed under a treatment	id, treatment_id, name, morning, afternoon, night, duration_days, instructions
DoctorAvailability	Doctor availability by date & slot	id, doctor_id, date, slot, enabled

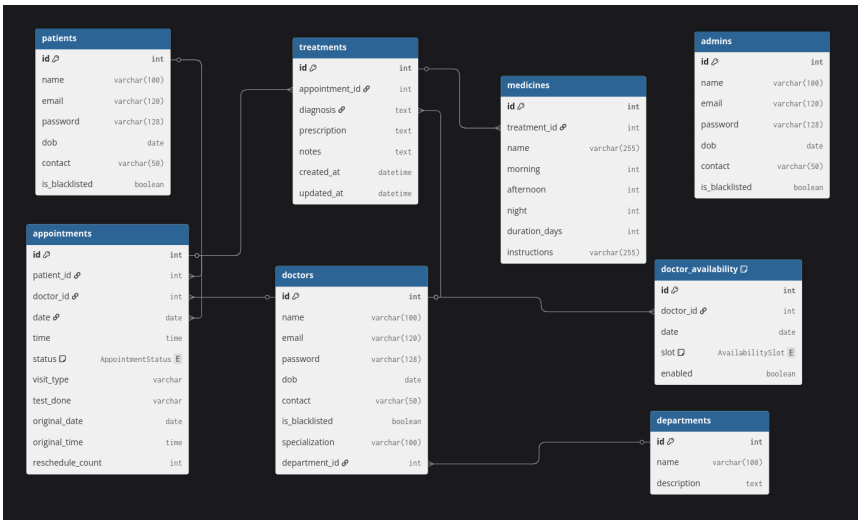
Relationships

One-to-Many

- **Department → Doctor**
  (One department has many doctors)
- **Doctor → Appointment**
  (One doctor can have many appointments)
  Patient → Appointment
  (One patient can have many appointments)
- **Treatment → Medicine**
  (One treatment can have many medicines)
- **Doctor → DoctorAvailability**
  (One doctor can have many availability entries)

One-to-One

- **Appointment → Treatment**
  (Each appointment has exactly one treatment record)



6. API Resource Endpoints

The system exposes REST APIs for login, booking and managing appointments, updating doctor availability, searching records, and admin-level user management. All endpoints are implemented as Flask Blueprints, use SQLAlchemy models for data access, and apply session and role checks through custom decorators.

Blueprints

- `api_bp` – All JSON API endpoints (booking, cancel, reschedule, availability, search, etc.).
- `auth_login_bp` – Login, logout.
- `auth_signup_bp` – Patient signup.
- `dashboard_bp` – Main dashboard for all roles.
- `admin_bp` – Admin functions (manage doctors/patients).
- `doctor_bp` – Doctor functions (history, profile, availability).
- `patient_bp` – Patient functions (booking, profile, departments).

7. Architecture and Features

Architecture

- **app.py** – Main Flask entry point; initializes app, database, and registers Blueprints.
- **models/** – Contains all SQLAlchemy ORM models (Admins, Doctors, Patients, Appointments, Treatments, Medicines, Availability).
- **routes/** – Controller layer organized into Blueprints:
    - `auth.py` → login/signup
    - `dashboard.py` → role-specific dashboards
      `api_routes.py` → JSON API endpoints
    - `services/doctor_service.py` → business logic helper

- **templates/** – Jinja2 templates grouped by module:
    - `admin/`, `doctor/`, `patient/` pages
    - `components/` for reusable modals (doctor modal, availability modal)
- **static/** – All frontend assets (CSS, JS, images).
- **utils/** – Constants, helper functions, setup scripts, and sample data generator.

Implemented Features

Admin

- Pre-existing admin created programmatically
- Add / update / delete doctor profiles
- View all appointments (upcoming + past)
- Search doctors/patients
- Blacklist/Unblacklist doctors or patients
- Dashboard showing total doctors, patients, appointments

Doctor

- View today’s and week’s appointments
- View all assigned patients
- Mark appointments as Completed/Cancelled
- Add diagnosis, prescriptions, and notes
- Provide availability for next 7 days (whole week)
- View full medical history of each patient

Patient

- Register, login, update profile
- View departments and doctor specializations
- View doctor availability for next 7 days
- Book, reschedule, or cancel appointments
- View upcoming appointments
- Access full medical/treatment history

Other Functionalities

- Prevent double-booking for same doctor/date/time
- Real-time appointment status updates
- Search by specialization, doctor name, patient name, or contact
- All tables created entirely using Python models (no manual DB editing)

8. Video Presentation

Drive Link:
https://drive.google.com/file/d/1ZDCxOMj8JrhBLS4Pkon3TGyX3yW6LmHB/view?usp=sharing